

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №7
по дисциплине
Методы и средства защиты информации
по теме
«Программная реализация шифрования алгоритмом «Магма»»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №7

по теме «Программная реализация шифрования алгоритмом «Магма»»

Цель работы

Создать криптографическую систему шифрования данных на основе блочного алгоритма «Магма» по ГОСТ Р 34.12-2015 и проверить корректность шифрования и дешифрования.

Задание

Программно реализовать алгоритм работы шифра «Магма» (шифрование и дешифрование). Для шифрования использовать определения из лабораторной работы 6.

Описание алгоритма

«Магма» — симметричный блочный шифр с размером блока 64 бита и ключом 256 бит. Исходный ключ разбивается на восемь 32-битных подключей. В раундах 1–24 подключи используются три раза в прямом порядке, в раундах 25–32 — в обратном.

Блок делится на левую и правую 32-битные половины. На каждом раунде к правой половине по модулю 2^{32} прибавляется подключ, затем восемь 4-битных частей заменяются по фиксированным S-блокам, а результат циклически сдвигается влево на 11 бит.

Полученное значение складывается по XOR с левой половиной; половины переставляются. В последнем раунде перестановка половин не выполняется. Расшифрование использует те же преобразования с обратным порядком подключей.

Схема раунда шифрования

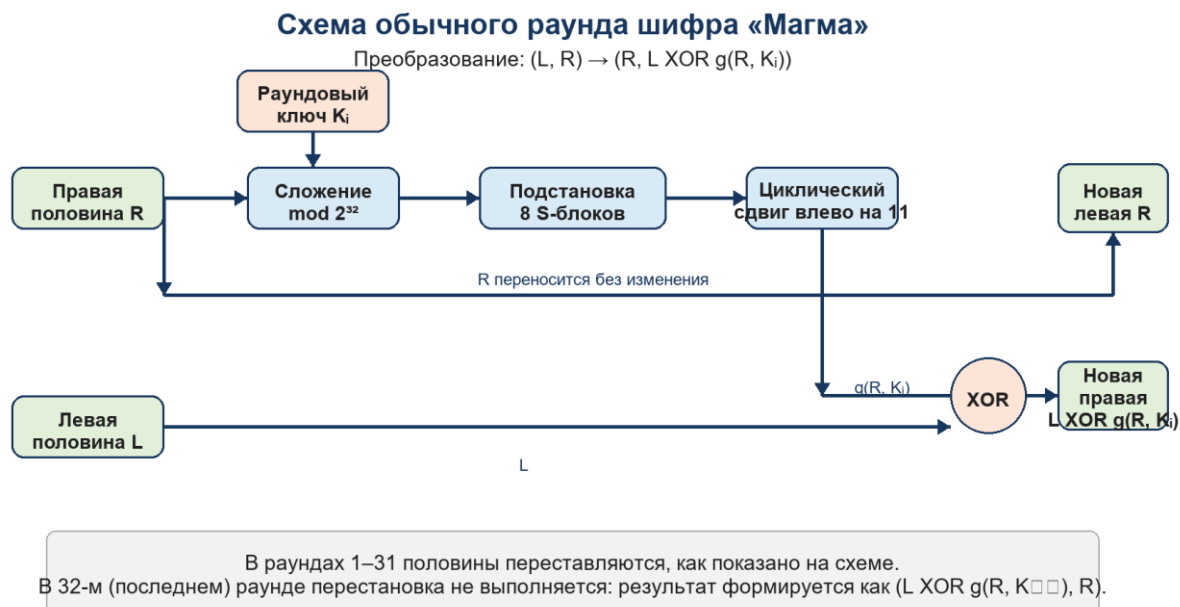


Рисунок 1 — Схема обычного раунда шифра «Магма»

Реализация

Программа `magma_lab.py` реализует S-преобразование, функцию g , расписание 32 раундовых ключей, шифрование и дешифрование одного блока. Для произвольного сообщения применяется PKCS#7-дополнение до размера блока. Исходное сообщение и шифртекст сохраняются в файлы `decrypted_message.txt` и `encrypted_message.hex` соответственно.

Результат выполнения

```

C:\Users\PC\Documents\ТулГУ\6_семестр\Методы_и_средства_защиты>
LP7: реализация алгоритма Магма завершена
Контрольный вектор ГОСТ Р 34.12-2015: пройден
Открытый блок: FEDCBA9876543210
Зашифрованный: 4EE901E5C2D8CA3D
Расшифрованный: FEDCBA9876543210
Длина сообщения: 100 байт
Длина шифртекста: 104 байт
Шифртекст (hex): BF260DAB7BAD4237FAD063304414DD377061DE13FB1623947C176BE886AE5F902B0CCBDAD5A26C28396EE55166CA8BA3F6E49F
023991E15C6C5A4CD4ACA0C556AE062CF4AC6523CE86989C70350F931A97EBB21F6A6505D99069A81C8B28F8596EA3DD0EAD20286
Проверка расшифрования пользовательского сообщения: пройдена
  
```

Рисунок 2 — Фактический запуск программы, контрольный вектор ГОСТ и проверка расшифрования

Для проверки использован контрольный вектор RFC 8891 (ГОСТ Р 34.12-2015): при ключе FFEEDDCC...FCFDFEFF и открытом блоке FEDCBA9876543210 получен шифртекст 4EE901E5C2D8CA3D.

Результат совпал с эталоном. Дополнительно зашифровано и расшифровано UTF-8-сообщение длиной 100 байт; его шифртекст имеет длину 104 байта из-за PKCS#7-дополнения.

Листинг программы

```
"""ЛР7: реализация блочного шифра «Магма» (ГОСТ 34.12-2015)."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

MASK32 = 0xFFFFFFFF
BLOCK_SIZE = 8
KEY_SIZE = 32

S_BOXES = (
    (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),
    (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15),
    (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),
    (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),
    (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),
    (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),
    (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),
    (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2),
)

def rotate_left_11(value: int) -> int:
    return ((value << 11) | (value >> 21)) & MASK32

def substitute(value: int) -> int:
```

```

        """Заменить восемь 4-битных частей значениями S-блоков
        Магмы."""
        result = 0
        for index, s_box in enumerate(S_BOXES):
            result |= s_box[(value >> (4 * index)) & 0xF] << (4 *
index)
        return result

def g(value: int, key: int) -> int:
    return rotate_left_11(substitute((value + key) & MASK32))

@dataclass(frozen=True)
class Magma:
    """Шифр Магма с 256-битным ключом и 64-битным блоком."""

    key: bytes

    def __post_init__(self) -> None:
        if len(self.key) != KEY_SIZE:
            raise ValueError("Ключ Магмы должен содержать ровно
32 байта")

    @property
    def round_keys(self) -> list[int]:
        parts = [int.from_bytes(self.key[i:i + 4], "big") for i
in range(0, KEY_SIZE, 4)]
        return parts * 3 + list(reversed(parts))

    def encrypt_block(self, block: bytes) -> bytes:
        if len(block) != BLOCK_SIZE:
            raise ValueError("Блок Магмы должен содержать ровно
8 байт")
        left = int.from_bytes(block[:4], "big")
        right = int.from_bytes(block[4:], "big")
        for key in self.round_keys[:-1]:

```

```

        left, right = right, left ^ g(right, key)
    left ^= g(right, self.round_keys[-1])
    return left.to_bytes(4, "big") + right.to_bytes(4,
"big")

def decrypt_block(self, block: bytes) -> bytes:
    if len(block) != BLOCK_SIZE:
        raise ValueError("Блок Магмы должен содержать ровно
8 байт")
    left = int.from_bytes(block[:4], "big")
    right = int.from_bytes(block[4:], "big")
    keys = list(reversed(self.round_keys))
    for key in keys[:-1]:
        left, right = right, left ^ g(right, key)
    left ^= g(right, keys[-1])
    return left.to_bytes(4, "big") + right.to_bytes(4,
"big")

def pad(data: bytes) -> bytes:
    padding = BLOCK_SIZE - len(data) % BLOCK_SIZE
    return data + bytes([padding]) * padding

def unpad(data: bytes) -> bytes:
    if not data or len(data) % BLOCK_SIZE:
        raise ValueError("Недопустимая длина зашифрованных
данных")
    padding = data[-1]
    if not 1 <= padding <= BLOCK_SIZE or data[-padding:] !=
bytes([padding]) * padding:
        raise ValueError("Некорректное PKCS#7-дополнение:
неверный ключ или данные")
    return data[:-padding]

def encrypt_data(cipher: Magma, data: bytes) -> bytes:
    raw = pad(data)

```

```

        return b"".join(cipher.encrypt_block(raw[i:i + BLOCK_SIZE])
for i in range(0, len(raw), BLOCK_SIZE))

def decrypt_data(cipher: Magma, data: bytes) -> bytes:
    raw = b"".join(cipher.decrypt_block(data[i:i + BLOCK_SIZE])
for i in range(0, len(data), BLOCK_SIZE))
    return unpad(raw)

def main() -> None:
    # Контрольный пример из ГОСТ Р 34.12-2015 / RFC 8891.
    test_key =
bytes.fromhex("FFEEDDCCBBA99887766554433221100F0F1F2F3F4F5F6F7F
8F9FAFBFCFDFEFF")
    plain_block = bytes.fromhex("FEDCBA9876543210")
    expected_block = bytes.fromhex("4EE901E5C2D8CA3D")
    test_cipher = Magma(test_key)
    encrypted_block = test_cipher.encrypt_block(plain_block)
    assert encrypted_block == expected_block, "Контрольный
вектор ГОСТ не совпал"
    assert test_cipher.decrypt_block(encrypted_block) ==
plain_block

    key =
bytes.fromhex("0123456789ABCDEFFEDCBA987654321000112233445566778
899AABBCCDDEEFF")
    cipher = Magma(key)
    message = "Конфиденциальное сообщение: Магма работает
корректно.".encode("utf-8")
    encrypted = encrypt_data(cipher, message)
    decrypted = decrypt_data(cipher, encrypted)
    assert decrypted == message

    root = Path(__file__).resolve().parent
    (root /
"encrypted_message.hex").write_text(encrypted.hex().upper() +
"\n", encoding="ascii")
    (root / "decrypted_message.txt").write_bytes(decrypted)

```

```

print("ЛР7: реализация алгоритма Магма завершена")
print("Контрольный вектор ГОСТ Р 34.12-2015: пройден")
print(f"Открытый блок:      {plain_block.hex().upper()}")
print(f"Зашифрованный:      {encrypted_block.hex().upper()}")
print(f"Расшифрованный:
{test_cipher.decrypt_block(encrypted_block).hex().upper()}")
print(f"Длина сообщения:  {len(message)} байт")
print(f"Длина шифртекста:  {len(encrypted)} байт")
print(f"Шифртекст (hex):    {encrypted.hex().upper()}")
print("Проверка расшифрования пользовательского сообщения:
пройдена")

if __name__ == "__main__":
    main()

```

Вывод

В ходе работы реализован блочный шифр «Магма» по ГОСТ Р 34.12-2015 без использования криптографических библиотек. Реализация выполняет шифрование и дешифрование блоков, корректно обрабатывает сообщения произвольной длины и подтверждена официальным контрольным вектором стандарта.

Список литературы

1. ГОСТ Р 34.12-2015. Информационная технология. Криптографическая защита информации. Блочные шифры.
2. RFC 8891. GOST R 34.12-2015: Block Cipher «Magma», 2020.